Highlights

- Evaluates procedure selection vs. workflow structuring in local RAG.
- Controlled ablation isolates document selection from graph serialization.
- No detected factual accuracy gain from graph structuring over raw text ($p = 0.59$).
- Workflow graphs prevent step-order inversions on local edge generators.
- Achieves 0.955 recall with 44% prompt-token reduction vs. full context.

Procedure Selection versus Workflow Structuring in Local RAG for Institutional SOP Question Answering: An Ablation Study

Cipta Fikri Wiratama[a], Indra Waspada[a,*], Dhena Kamalia Fu'adi[a], Ramona Ramli[b,*], Henri Tantyoko[a], Sandy Kurniawan[a], Satriawan Rasyid Purnama[a]

[a] *Department of Informatics, Faculty of Science and Mathematics, Universitas Diponegoro, Semarang, 50275, Indonesia*
[b] *School of Computer Sciences, Universiti Sains Malaysia, Pulau Pinang, 11800 USM, Malaysia*

Abstract

Standard operating procedures (SOPs) in university administration govern sequential processes requiring strict adherence to step order, actor responsibilities, and required documentation. Deploying locally hosted language models circumvents commercial API fees and protects data privacy, but smaller models struggle to resolve distractor passages retrieved from lexically overlapping procedures. We investigate a workflow-aware Retrieval-Augmented Generation (RAG) pipeline that pairs rule-based procedure selection with directed workflow graphs encoding chronological steps and responsible actors. We evaluate the architecture against nine alternative configurations on a verified 40-question benchmark across seven Indonesian institutional SOPs using a local generator (`gemma4:e2b`, 5.1B parameters). The complete Workflow GraphRAG pipeline achieves 0.955 key-fact recall (KFR) and 1.000 step coverage, improving upon naive RAG (0.712 KFR; Holm-adjusted $p = 0.053$) and a custom entity-graph baseline (0.698 KFR; Holm-adjusted $p = 0.034$). However, a selection-matched text ablation providing the identical selected SOP as raw text achieves 0.962 KFR; a paired Wilcoxon test detects no statistically significant factual recall advantage from graph serialization ($p = 0.59$). Furthermore, non-fine-tuned base embeddings elevate naive RAG to 0.910 KFR, revealing strong baseline sensitivity. In procedural ordering, Workflow GraphRAG maintains zero topological inversions (TIR = 0.000), whereas entity-graph retrieval (TIR = 0.083) and full-context prompting (TIR = 0.027) introduce ordering errors. Compared to full-context prompting, workflow graphs reduce prompt tokens by 44% (1,525 vs. 2,722 tokens) while preserving comparable recall. These results demonstrate that procedure-level selection is the primary driver of factual recall in concise procedural corpora, while workflow graphs provide chronological formatting in-

*Corresponding author
Email addresses: `fikriw@students.undip.ac.id` (Cipta Fikri Wiratama), `indrawaspada@lecturer.undip.ac.id` (Indra Waspada), `ramona@usm.my` (Ramona Ramli)

tegrity and token efficiency.

Keywords: Retrieval-augmented generation, GraphRAG, Standard operating procedures, Workflow representation, Small language models, Ablation study, Evaluation metrics

---

1. Introduction

Academic administration in higher education institutions relies heavily on Standard Operating Procedures (SOPs) to govern repetitive workflows, including academic leave applications, course registration, tuition payment postponements, and credential verification. Although these procedures are formally documented in official manuals, administrative staff spend substantial time repeatedly addressing student inquiries regarding required steps, responsible administrative units, and prerequisites. Automated question answering systems based on Large Language Models (LLMs) represent a viable mechanism to mitigate this administrative overhead.

However, operationalizing automated assistants within public universities involves critical resource and governance constraints. Relying on commercial, cloud-hosted proprietary language models introduces cumulative per-query monetary costs and exposes internal institutional records and student identity data to third-party infrastructure. Executing small, open-weights language models entirely on local, on-premise hardware resolves privacy, data residency, and subscription expenditure concerns. Nonetheless, parameter-constrained local models (such as 2B to 5B parameter architectures) exhibit reduced intrinsic reasoning capabilities, heightened vulnerability to distractors, and limited context synthesis compared to multi-billion parameter cloud services [10]. In administrative question answering, factual correctness must be preserved strictly: an automated response that transposes chronological steps, assigns a verification task to the wrong university official, or misidentifies mandatory clearance forms causes direct operational disruptions and sends students to incorrect administrative desks.

Retrieval-Augmented Generation (RAG) grounds language model generations in external source corpora by retrieving relevant text passages prior to inference [1, 2]. Standard passage retrieval splits documents into fixed-length segments and retrieves top-k nearest neighbors based on semantic vector similarity. For institutional procedures, this conventional mechanism presents specific failure modes. When documents are concise, standard chunking often results in whole-document retrieval units. Consequently, standard top-k dense retrieval frequently returns multiple distinct SOPs containing overlapping administrative vocabulary (e.g., Dean, Academic Office, verification, approval signatures). A small generator must therefore disentangle multiple competing procedural streams, risking procedural cross-contamination and hallucinated step orderings.

To enhance context quality, graph-based RAG architectures have been proposed [4, 11]. Mainstream GraphRAG methodologies typically construct knowl-

edge graphs centered on named entities and relational semantic triples [3, 12, 13]. While effective for multi-hop entity discovery and thematic summarization, entity-centric graphs do not naturally represent sequential execution flow, temporal precedence, or step-contingent actor responsibility.

To address sequential procedural grounding, workflow-aware approaches structure procedures as directed execution graphs [16]. In this work, we investigate *Process-Aware Workflow GraphRAG*, an architecture designed for local procedural question answering. The system models linear institutional SOPs as directed workflow graphs where step nodes explicitly capture actor responsibility, procedural actions, required documentation, and resulting deliverables, connected by sequential precedence edges. Combined with a transparent rule-based SOP selection mechanism and supporting text retrieval, the pipeline generates structured procedural prompts for a local generator (`gemma4:e2b`).

Critically, evaluating graph-enhanced retrieval requires disentangling architectural components. In many existing graph-based RAG studies, retrieval accuracy gains are attributed broadly to graph modeling without isolating whether the improvement originates from *retrieving the correct source document* or from *serializing evidence as an explicit graph structure*. If selecting the appropriate procedure is the primary bottleneck of standard RAG, providing the raw text of that selected procedure might achieve comparable factual accuracy without requiring graph curation.

To rigorously investigate this empirical question, we formulate four research questions:

- **RQ1 (Answer Quality and Completeness):** How does the workflow-aware RAG pipeline compare with conventional chunk retrieval and custom entity-graph baselines in factual completeness, step coverage, and procedural order and actor correctness?
- **RQ2 (Selection vs. Structuring):** What measurable contribution, if any, does explicit graph-derived workflow structuring provide beyond selecting the identical SOP and providing its original text?
- **RQ3 (Refusal and Boundary Behavior):** How do the evaluated systems behave when presented with out-of-scope or unanswerable questions, and what mechanisms govern their refusal decisions?
- **RQ4 (Efficiency and Token Trade-offs):** What are the observed prompt-token, latency, and computational trade-offs of the workflow-aware approach compared with conventional retrieval baselines and full-context prompting?

This study makes four primary contributions:

1. **Workflow-Aware Procedural RAG Pipeline:** We present a workflow-grounded retrieval architecture that combines rule-based SOP identification with directed workflow graph serialization to provide ordered step sequences and explicit actor attributions to a locally hosted small generator.

2. **Controlled Selection-versus-Structure Ablation:** We introduce a selection-matched text ablation (*Selection-Matched Text RAG*, designated in benchmark logs as *SOP-Oracle RAG*) that holds procedure selection and supporting chunk retrieval identical while replacing the graph-structured block with raw SOP text, isolating document selection from structural representation.

3. **Deterministic Procedural Evaluation Metrics:** We employ two deterministic, LLM-free evaluation metrics—Topological Inversion Rate (TIR) for step-order violations and Actor Mismatch Rate (AMR) for actor attribution errors—complemented by Step Coverage and Key-Fact Recall (KFR), validated through a manual audit of 60 sampled rows.

4. **Empirical Analysis of Comparative Nuances:** We conduct a controlled empirical comparison across 10 systems on a verified 40-question institutional benchmark. We demonstrate that while Workflow GraphRAG achieves strong factual recall (0.955), the selection-matched text ablation achieves comparable performance (0.962; paired Wilcoxon $p = 0.59$). The findings are consistent with the importance of procedure-level evidence selection, although the design does not fully isolate its causal contribution from associated retrieval and prompting differences. Furthermore, we reveal that baseline embedding configurations substantially influence standard RAG competitiveness, establishing critical boundaries for graph-based interventions in procedural NLP.

The remainder of this paper is structured as follows. Section 2 reviews related work. Section 3 defines the institutional setting and procedural error classes. Section 4 details the workflow graph architecture and selection ablation. Section 5 describes the experimental setup, benchmark, and metrics. Section 6 presents the empirical results. Section 7 analyzes threats to validity and architectural limitations. Section 8 concludes and outlines future research.

2. Related work

2.1. Retrieval-augmented generation

Retrieval-Augmented Generation (RAG) mitigates parametric hallucination by conditioning text generation on retrieved external documents [1]. Conventional naive RAG partitions a corpus into discrete text chunks, indexes them with dense vector embeddings, and injects the top-k nearest neighbors into the generator prompt [2]. To improve retrieval quality, hybrid architectures combine dense vector search with sparse lexical scoring (such as BM25 [5]) using reciprocal rank fusion [6]. Iterative and agentic RAG frameworks, including ReAct [7], allow the model to interleave reasoning traces with external tool executions. Self-reflective methods, such as Corrective RAG (CRAG) [8] and Self-RAG [9], evaluate retrieval confidence and execute query rewriting or fallback web searches when evidence is evaluated as insufficient.

Despite these retrieval innovations, conventional text-based RAG architectures consistently provide generators with unstructured prose passages. In procedural domains, extracting sequential execution order and matching actions to responsible actors remains an implicit reasoning task delegated entirely to the generator. In long-context language models, supplying entire document collections within the prompt offers an alternative to retrieval, but introduces increased prompt-token costs and risks degraded attention over middle context regions [10].

2.2. Graph-based RAG architectures

Graph-based RAG incorporates structured relationships to overcome the limitations of isolated text chunks [4, 11]. Microsoft GraphRAG [3] extracts entity-relation networks using LLM prompting, clusters nodes into hierarchical communities, and generates summaries to address global, multi-document thematic queries. LightRAG [12] introduces dual-level entity and relationship indexing to reduce computational overhead. HippoRAG [13] models associative memory through Personalized PageRank over entity graphs to facilitate multi-hop passage retrieval.

These frameworks primarily model relational triples among named entities (e.g., entity A connected to entity B via relation R). However, conventional entity graphs lack native formalisms for strict chronological precedence, conditional control branching, or sequential execution states required for standard operating procedures.

2.3. Process modeling and procedural representation

The representation of structured procedures has long been established in Business Process Management (BPM) [16]. The Business Process Model and Notation (BPMN) standardizes directed workflow modeling using activity tasks, sequence flows, swimlanes for actor roles, and gateways for conditional branching [17]. Prior computational linguistic studies have investigated extracting formal process models directly from natural language process descriptions [18]. While process modeling typically targets formal business simulation, compliance verification, and software workflow orchestration, this study adapts workflow graphs as an information retrieval and prompt structuring substrate for language model generation.

2.4. Emerging order-aware and procedural RAG

Recent concurrent investigations have recognized the necessity of explicitly modeling procedural order. Lin et al. [14] introduced SOPRAG, an architecture designed for industrial SOP retrieval employing dual-view graph experts: flow graph experts capturing step sequences and entity graph experts capturing associated tools and operational components. Similarly, Wu et al. [15] proposed Order-Aware Hypergraph RAG, utilizing transition modeling over hyperedges to capture chronological order across multi-step interactions.

Our investigation differs in scope and objective. While SOPRAG and Hypergraph RAG focus on large-scale industrial procedures and multi-hop graph neural indexing, our study focuses on small, locally hosted language models operating over institutional administrative workflows. More fundamentally, prior studies evaluate complex architectures without isolating the specific contribution of procedure-level selection from structured graph serialization. This paper addresses this methodological gap by introducing an ablation design that directly separates document selection from structural representation.

Table 1: Structural characteristics of context units provided to the generator across retrieval paradigms. "Implicit" denotes that information is present in prose but requires model inference.

Method family	Context unit passed to generator	Step sequence	Actor per step
Naive / Hybrid / Agentic / CRAG [1, 6, 7, 8]	Unstructured text chunks	Implicit	Implicit
Entity-graph GraphRAG [3, 12]	Entity-relation triples & text	Not represented	Relational triples
Workflow GraphRAG (this work)	Directed workflow block & chunks	Explicit	Explicit
Selection-Matched Text (ablation)	Whole selected SOP text & chunks	Implicit	Implicit

3. Problem setting and procedural error classes

3.1. Institutional SOP corpus

The empirical setting comprises the complete set of official student-service Standard Operating Procedures published by the Faculty of Science and Mathematics (FSM), Universitas Diponegoro (Table 2). The corpus consists of 7 one-page Indonesian administrative documents governing academic standing, tuition processing, and student activities. Each procedure is structured as a strictly linear sequence of numbered administrative steps with no decision gateways or concurrent branches. Across the 7 documents, the corpus contains 44 numbered steps and approximately 1,097 words.

3.2. Procedural correctness requirements and error taxonomy

A satisfactory administrative answer must fulfill strict correctness criteria: it must enumerate mandatory steps in execution order, attribute each action to the precise administrative official, identify prerequisites, and state resulting deliverables. We formally define three specific failure modes:

- **E1 (Step Order Inversion):** The generated response transposes execution order, presenting downstream actions prior to prerequisite clearances (e.g., presenting Dean approval before Academic Sub-division verification).

Table 2: Characteristics of the 7 FSM Universitas Diponegoro SOP documents. "PDF steps" denotes numbered steps in official source documents; "Graph steps" denotes nodes in the curated workflow graph.

SOP Document	PDF steps	Graph steps	Characters	Words
Academic leave (Cuti Akademik)	8	8	1,573	205
Reactivation after leave (Aktif Kembali)	3	3	597	84
Diploma/transcript legalisation (Legalisir)	6	6	1,640	208
Course registration (Pengisian IRS)	6	6	1,189	163
Late tuition payment (Keterlambatan UKT)	11	11	1,415	173
Scholarship recommendation (Beasiswa)	5	5	1,130	142
Student activity proposal (Proposal Ormawa)	5	5	997	122
Total	44	44	8,541	1,097

- **E2 (Actor Mismatch):** The response assigns a procedural task to an incorrect official or office (e.g., claiming the Dean reviews application attachments when verification is executed by Academic Sub-division staff).
- **E3 (Procedural Cross-Contamination):** The response incorporates procedural steps or prerequisites from an irrelevant SOP into the requested workflow (e.g., introducing tuition settlement steps into an academic leave inquiry).

3.3. Corpus chunking and procedural cross-contamination

Under standard RAG implementations, text documents are partitioned into fixed token or character windows. In our experimental framework, documents are chunked with a window of 1,700 characters and an overlap of 100 characters. Because each SOP in Table 2 is under 1,650 characters, each document maps onto exactly one chunk in the vector index ($N = 7$).

When conventional RAG retrieves top-k nearest neighbors ($k = 3$), the retrieved context necessarily contains three distinct, complete SOPs. Because Indonesian university administrative procedures share high lexical overlap—frequently citing forms, faculty leadership, academic sub-divisions, verification, and signatures—retrieving three distinct procedures creates high distraction for small language models. The challenge in this institutional setting is not reconstructing fragmented passages, but preventing procedural cross-contamination (E3) and preserving strict chronological and actor fidelity (E1, E2).

4. Proposed method and ablation design

4.1. System overview

Process-Aware Workflow GraphRAG combines an offline structured curation phase with an online multi-stage retrieval and synthesis pipeline. Figure 1 illustrates the complete workflow. During offline preparation, source SOPs are curated into directed workflow graphs, and text passages are indexed in a dense vector store. During online query processing, the system selects the target SOP via rule-based scoring, serializes its ordered steps into a workflow text block, retrieves supporting document chunks, and prompts a local generator.

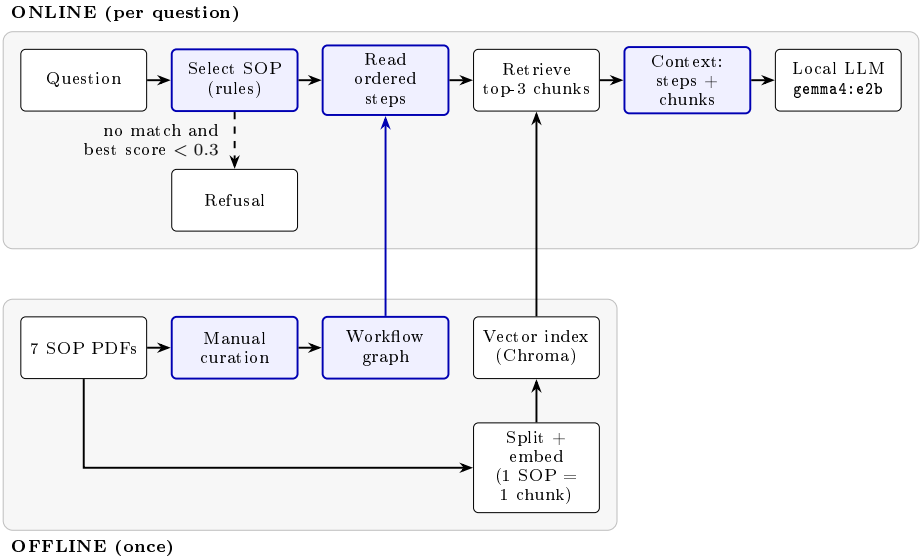


Figure 1: Architecture of Process-Aware Workflow GraphRAG. Highlighted nodes indicate components introduced: rule-based SOP selection and ordered workflow graph extraction. The underlying vector index, embedding model, prompt template, and generator are shared across evaluated systems.

4.2. Workflow graph construction

Each procedural document p is represented as a directed graph $G_p = (V_p, E_p)$ implemented via NetworkX. The node set V_p comprises three distinct classes:

1. **Root Node (v_{root}):** Represents the complete procedure, storing the official title, administrative aliases (e.g., "cuti kuliah" for academic leave), and official maximum execution duration where specified.

2. **Step Nodes (v_i):** Encapsulates individual procedural steps $i \in \{1, \ldots, n_p\}$. Each node stores: the primary performing actor; action description; required input documents; resulting deliverables; duration; and a verbatim textual quote from the source PDF.

3. **Actor Nodes (v_{actor}):** Represents distinct organizational roles or administrative offices shared across procedures.

Edges E_p capture three relationship types:

- `HAS_STEP`(v_{root}, v_i): Links the procedure root to each constituent step.
- `NEXT_STEP`(v_i, v_{i+1}): Encodes chronological execution precedence between consecutive steps.
- `PERFORMS`(v_{actor}, v_i): Attributes operational execution of step v_i to a specific actor node.

The workflow graph was constructed by hand from the 7 official PDFs, producing 7 root nodes, 44 step nodes, and 15 actor nodes connected by 125 directed edges. Steps were defined such that the primary actor corresponds to the operational performing entity (the grammatical subject of the procedural instruction), whereas consulted or secondary signatories are recorded within the action description. Automated verification scripts verified that every step quote matches the source text verbatim and that step counts align exactly with source PDFs.

4.3. Rule-based SOP selection

Given user question q, the system identifies the target procedure p^* using a transparent heuristic scoring function:

$$s(q,p) = 15\mathbb{1}[title_p \subset q] + \sum_{a \in aliases_p} g(q,a) + \sum_{v \in steps_p} h(q,v), \tag{1}$$

where $g(q,a) = 8$ if alias a occurs as an exact substring in q, or 2 per constituent word ($|w| > 2$); and $h(q,v) = 1$ if q contains an explicit procedural cue (Indonesian keywords for "how", "procedure", "requirements", "flow", or "documents") and shares a content word ($|w| > 4$) with the step action description.

The procedure with the maximum score is selected if $s(q,p^*) \geq 3$. A deterministic priority rule maps questions simultaneously containing keywords for "active" and "leave" to *Reactivation after leave*, resolving potential confusion between the two procedures. Matching is case-insensitive. While rule-based matching is deterministic and inspectable, it is tailored to this 7-SOP collection.

4.4. Context construction and supporting retrieval

When procedure p^* is selected, its step nodes are extracted via a topological traversal along `NEXT_STEP` edges. The steps are serialized into an ordered textual context block specifying step index, performing actor, action, required documents, outputs, and duration (Figure 2).

To provide access to supplementary document details (such as administrative notes and URLs), the query is expanded by concatenating q with the SOP title and actions from its first three steps. Dense retrieval queries the shared Chroma vector index using the expanded query, retrieving $k = 3$ supporting chunks that are appended after the workflow block. Because each chunk in the vector index corresponds to one SOP, these supporting chunks may contain text from other SOPs, which we analyze in Section 7.

4.5. Generation and refusal mechanism

Context and question are injected into a standard prompt template instructing the model to respond in Indonesian, adhere strictly to provided context facts, and abstain if evidence is insufficient. Inference is executed locally using `gemma4:e2b` via Ollama.

OFFICIAL WORKFLOW: REACTIVATION AFTER LEAVE (AKTIF KEMBALI)
Maximum processing duration: Not specified in document

Step 1. Actor: Mahasiswa (Student)
Action: Submit academic leave permit letter from prior semester and student card copy to Head of Study Programme and Academic Sub-division.
Required Documents: Academic leave permit letter, Student card (KTM) copy
Duration: ±5 minutes

Step 2. Actor: Mahasiswa (Student)
Action: Register online for reactivation through university Single Sign-On (SSO) portal.
...

Step 3. Actor: Mahasiswa (Student)
Action: Registered as active course participant in academic database.
...

Supporting Documents: *[Top-3 dense chunks retrieved from vector store]*

Figure 2: Illustrative context block produced for reactivation inquiry (translated from Indonesian). Each step explicitly details performing actor, action, and required documentation.

Refusal operates through a combination of deterministic pre-generation filtering and model-generated abstention. Deterministic refusal triggers before generator invocation if: (1) no SOP satisfies selection ($s(q, p^*) < 3$); and (2) maximum cosine similarity of q against retrieved vector chunks is below threshold $\tau = 0.3$. If an SOP is selected, the query proceeds to the generator, which can still produce a refusal response if the provided context lacks the requested facts. Algorithm 1 details the execution pipeline.

4.6. Selection-matched text ablation design

A central methodological question is whether observed performance gains stem from explicit workflow graph structuring or from procedure selection. To isolate these factors, we implement *Selection-Matched Text RAG* (designated in experimental benchmark logs as *SOP-Oracle RAG*).

This ablation shares the identical SOP selection rule (Eq. (1)) and query-expanded supporting chunk retrieval as Workflow GraphRAG. However, instead of serializing the workflow graph W, it retrieves the raw, unedited text chunk of the selected SOP p^* from the document store. Thus, both systems receive the identical procedure and supporting chunks, differing solely in representation: structured workflow block vs. original unstructured prose.

5. Experimental setup

5.1. Evaluated systems

We evaluate 10 system configurations grouped into three functional categories (Table 3):

Algorithm 1 Execution pipeline of Process-Aware Workflow GraphRAG

Require: Question q, workflow graph G, vector index I, threshold $\tau = 0.3$, retrieval depth $k = 3$

$p^* \leftarrow \arg\max_p s(q, p)$ {Evaluate selection via Eq. (1)}
if $s(q, p^*) < 3$ **then**
  $p^* \leftarrow$ none
end if
if $p^* \neq$ none **then**
  $W \leftarrow$ serialized ordered step sequence of p^* from G
  $q' \leftarrow q \,\|\, title_{p^*} \,\|\, actions_{1:3}(p^*)$ {Query expansion}
else
  $W \leftarrow \emptyset$
  $q' \leftarrow q$ {Retain unexpanded query}
end if
$C \leftarrow \textsc{RetrieveTopK}(I, q', k)$
$b \leftarrow \max_{c \in C} \textsc{CosineSimilarity}(q, c)$
if $p^* =$ none **and** $b < \tau$ **then**
  return Refusal response ("Information not available in official SOPs")
end if
return $\textsc{GenerateLocalLLM}(\textsc{PromptTemplate}(W \,\|\, C, q))$

1. **Main Comparison Systems:**

   - *Naive RAG:* Standard baseline retrieving top-3 dense chunks. Refuses if top similarity < 0.3.
   - *Custom Entity-Graph GraphRAG:* Baseline matching question entities against a hand-curated undirected graph of 17 relational triples (e.g., procedure linked to responsible office or required document), expanding to 1-hop neighbors, appending relation triples to prompt, and retrieving top-3 chunks.
   - *Workflow GraphRAG (Proposed):* Full architecture combining rule-based SOP selection, ordered workflow graph serialization, and supporting chunk retrieval.
   - *Selection-Matched Text RAG (Ablation):* Identical selection and supporting chunks as Workflow GraphRAG, supplying raw SOP text in place of graph serialization.

2. **Supplementary RAG Baselines:**

   - *Hybrid RAG:* Dense top-5 and BM25 top-5 retrieval merged via Reciprocal Rank Fusion ($k_{\text{rrf}} = 60$), retaining top-3 chunks.
   - *Agentic RAG:* Multi-turn ReAct reasoning loop allowing up to 3 iterative search actions prior to generation.
   - *Corrective RAG (CRAG):* Evaluates retrieval confidence; scores ≥ 0.55 proceed directly, 0.35–0.55 trigger LLM query rewriting, < 0.35 are refused.

- *Multimodal-Layout RAG:* Dense retrieval incorporating structural document cues (table and section tags) extracted during ingestion.

3. **Reference Systems:**

- *LLM Only:* Parametric generation without retrieval, establishing base generator knowledge.
- *Full-Context Prompting:* Supplies all 7 SOP documents (≈1,100 words) in the prompt, representing an upper bound on available evidence without retrieval filtering.

Table 3: System configurations evaluated across 10 architectures. All systems share identical embedding models, chunking parameters (1,700/100 characters), vector stores, generator, and prompt templates.

System Configuration	Context Construction and Retrieval Mechanism
Main Comparison and Ablation	
Naive RAG	Top-3 dense chunks by cosine similarity; refuses if best similarity < 0.3.
GraphRAG (custom entity graph)	1-hop expansion over 17 curated relational triples; matched triples prepended to top-3 dense chunks.
Workflow GraphRAG (proposed)	Ordered workflow steps from directed graph with explicit actors and inputs, followed by top-3 supporting chunks.
Selection-Matched Text RAG (SOP-Oracle ablation)	Identical rule-based selection, providing selected raw SOP text in place of workflow block, followed by top-3 supporting chunks.
Supplementary RAG Baselines	
Hybrid RAG	Reciprocal rank fusion of top-5 dense and top-5 BM25 chunks ($k_{\text{rrf}} = 60$); top-3 retained.
Agentic RAG	ReAct loop executing up to 3 iterative search queries directed by generator.
Corrective RAG (CRAG)	Retrieval grading: ≥ 0.55 retained, 0.35–0.55 query rewritten, < 0.35 refused.
Multimodal-Layout RAG	Top-3 dense chunks augmented with structural document layout tags.
Reference Baselines	
LLM Only	Zero-shot generation without retrieved context.
Full-Context Prompting	All 7 SOP documents provided directly in generator prompt (≈1,100 words).

5.2. Custom entity-graph baseline scope and validity threat

We explicitly state that the entity-graph GraphRAG baseline is an author-developed implementation consisting of 17 curated relational triples. It is not an official deployment of Microsoft GraphRAG [3], LightRAG [12], or HippoRAG [13]. Because the baseline was constructed by the authors, it represents an empirical point of comparison for basic entity-triple augmentation rather than

an exhaustive evaluation of published GraphRAG frameworks. We treat this as an explicit validity constraint in Section 7.

5.3. Shared infrastructure and experimental controls

All retrieval systems share identical underlying infrastructure:

- **Embedding Model:** `all-indo-e5-small-v4` [19], an Indonesian sentence embedder (384 dimensions) fine-tuned on 50 procedural question-answer pairs using Multiple Negatives Ranking (MNR) loss [21]. An ablation evaluates the non-fine-tuned base model.
- **Vector Store:** Chroma vector database storing 7 chunks (1,700 character window, 100 character overlap).
- **Generator:** `gemma4:e2b` (5.1B total parameters, Q4_K_M quantization) served locally via Ollama 0.35.1 at temperature 0, seed 42, following one warm-up inference.
- **Hardware and Invocation Logging:** Apple M5 workstation (10 CPU cores, 24 GB unified memory) running macOS 26.6.2 on AC power. Every generator call records prompt tokens, completion tokens, and latency via unified logging wrappers. The mean number of generator invocations per query reflects deterministic pre-generation refusal: systems that filter out low-similarity queries prior to inference average fewer than 1.00 calls across the benchmark.

5.4. Evaluation benchmark

We developed a held-out benchmark of 40 questions drafted from the source SOPs (Table 4). The benchmark items were not generated by any evaluated system, and a lexical overlap audit confirmed no question shares ROUGE-L ≥ 0.5 with the fine-tuning training pairs.

The benchmark contains 35 answerable questions (comprising 32 single-SOP questions across procedural, actor, document, and time categories, plus 3 cross-SOP questions) and 5 unanswerable questions. Each answerable item defines a reference answer and required key facts. Each procedural item additionally specifies a gold sequence of execution steps, associated actors, and matching keyword groups. All items were verified against official PDFs by the first author.

5.5. Evaluation metrics

Evaluation metrics are computed deterministically without reliance on LLM judges:

- **Key-Fact Recall (KFR):** Proportion of required reference facts detected in the generated response, evaluated across the 35 answerable questions.
- **ROUGE-1, ROUGE-2, ROUGE-L:** Standard n-gram and longest common subsequence overlap against reference answers [22].

Table 4: Composition of the 40-question held-out institutional benchmark.

Category	n	Operational Target
Procedural	11	Complete step sequence, chronological order, and actors (7 formal, 4 colloquial).
Actor attribution	7	Identification of verifying, signing, or receiving officials.
Document requirements	7	Identification of mandatory forms, attachments, and outputs.
Time and location	7	Stated execution durations, service desks, and IT portals.
Cross-SOP discrimination	3	Differentiating easily confused procedures (evaluating E3).
Unanswerable / Refusal	5	Inquiries concerning out-of-scope administrative rules.
Total	40	

- **Step Coverage:** For procedural questions ($n = 11$), proportion of gold steps S detected in the generated response ($|D|/|S|$), where detection requires matching at least two keyword groups.
- **Topological Inversion Rate (TIR):** Proportion of detected step pairs (v_i, v_j) with $i < j$ that appear in reversed order in the response:

$$\mathrm{TIR} = \frac{\left|\{(v_i, v_j) \in D^2 : i < j,\ pos(v_i) > pos(v_j)\}\right|}{\left|\{(v_i, v_j) \in D^2 : i < j,\ pos(v_i) \neq pos(v_j)\}\right|}. \tag{2}$$

  TIR is conditional on detecting at least two comparable steps.
- **Actor Mismatch Rate (AMR):** Proportion of detected steps attributed to an incorrect official among steps where an actor is explicitly named:

$$\mathrm{AMR} = \frac{mismatch}{correct + mismatch}. \tag{3}$$

  AMR is conditional on step detection and explicit actor naming.
- **Refusal Accuracy and False Refusal Rate:** Refusal accuracy is the proportion of the 5 unanswerable questions correctly refused. False refusal rate is the proportion of the 35 answerable questions incorrectly refused.
- **Resource Costs:** Prompt tokens, completion tokens, LLM invocations, and elapsed latency per question, averaged across all 40 benchmark items.

5.6. Manual audit of deterministic step scorer

Because step detection, TIR, and AMR depend on keyword matching, we audited the deterministic scorer. A sample of 60 step-alignment rows was drawn across Naive RAG, GraphRAG, and Workflow GraphRAG (stratified, seed 42) and evaluated manually by the first author against full response texts (Table 5).

Table 5: Manual audit of deterministic step-alignment scorer across 60 sampled rows (Naive RAG, GraphRAG, and Workflow GraphRAG).

Measure	Agreement / κ	Labelled
Step detected	0.983 / 0.955	60/60
Actor status	0.933 / 0.835	60/60

The manual audit demonstrated 0.983 agreement ($\kappa = 0.955$) on step detection and 0.933 agreement ($\kappa = 0.835$) on actor attribution across the 60 sampled rows. The single step detection discrepancy occurred on GraphRAG for late tuition payment (UKT-P1, step 5), where the model formatted actor and action on separate markdown bullet lines ("**Pelaku:** Wakil Dekan Sumber Daya" followed by "**Tindakan:** Memproses..."). Because the scorer segments text line-by-line and requires two keyword groups per line, it recorded a non-detection despite both elements being present. Crucially, the manual audit confirmed zero genuine actor attribution errors across the 60 sampled rows (AMR = 0.000). We note that this audit validates scorer fidelity for the audited sample, but does not imply zero actor errors across all uninspected generated texts.

5.7. Secondary LLM-as-a-judge metrics

As a supplementary check, Ragas faithfulness and answer relevancy [23] were computed using an independent judge model (`llama3.1:8b`) [24]. Faithfulness evaluates claims against provided context (including workflow blocks). Due to occasional JSON parsing failures in the judge output (yielding valid scores for 35/40 questions in Workflow GraphRAG and 26–27/40 in baselines), Ragas metrics are treated strictly as secondary corroboration.

5.8. Statistical testing methodology

To account for benchmark sample size ($n = 40$), all metrics report sample means with 95% bootstrap confidence intervals (10,000 resamples) [25]. Comparisons between Workflow GraphRAG and baselines utilize two-sided paired Wilcoxon signed-rank tests [26]. Holm-Bonferroni correction [27] is applied within distinct hypothesis testing families (separately for the family of comparisons against Naive RAG and against GraphRAG).

We emphasize that bootstrap confidence intervals and Holm-adjusted hypothesis tests operate under distinct multiplicity assumptions: a confidence interval for a single pairwise difference may exclude zero while the corresponding family-wise adjusted p-value exceeds $\alpha = 0.05$. We report both transparently and avoid describing non-significant results as established equivalence.

6. Experimental results

Empirical evaluations are derived from the execution run on all 40 verified benchmark items (`results/full_v3_20261008/`, branch `exp/full-run-v1`).

6.1. Procedural answer quality and order preservation (RQ1)

Table 6 reports performance across all 10 evaluated systems. Workflow GraphRAG, Selection-Matched Text RAG, and Full-Context Prompting achieve top-tier performance on key-fact recall: 0.955 [0.917, 0.990], 0.962 [0.914, 1.000], and 0.962 [0.914, 1.000], respectively. In contrast, Naive RAG achieves 0.712 [0.571, 0.840] and custom entity-graph GraphRAG achieves 0.698 [0.560, 0.829]. Workflow GraphRAG attains 1.000 [1.000, 1.000] step coverage on procedural questions, compared to 0.727 for Naive RAG and 0.687 for GraphRAG.

Table 6: Summary of evaluation results across ten system configurations. Key-fact recall (KFR) and false refusal rate are evaluated on 35 answerable questions; procedural metrics (Coverage, TIR, AMR) on 11 procedural questions; refusal accuracy on 5 unanswerable questions; and prompt tokens on all 40 questions. Values are point estimates; full bootstrap 95% confidence intervals are reported in Appendix Appendix A. TIR: Topological Inversion Rate; AMR: Actor Mismatch Rate.

System	KFR ↑	Coverage ↑	TIR ↓	AMR ↓	Refusal acc. ↑	False ref. ↓	Prompt tok.
Naive RAG	0.712	0.727	0.000	0.000	0.600	0.000	916
GraphRAG (entity)	0.698	0.687	0.083	0.000	0.600	0.000	949
Workflow GraphRAG	0.955	1.000	0.000	0.000	1.000	0.000	1525
Selection-Matched Text RAG (ablation)	0.962	0.975	0.000	0.000	1.000	0.000	1479
Hybrid RAG	0.683	0.712	0.000	0.000	0.600	0.000	882
Agentic RAG	0.712	0.702	0.000	0.000	0.600	0.000	1815
Corrective RAG	0.569	0.636	0.000	0.000	1.000	0.371	725
Multimodal-layout RAG	0.702	0.702	0.000	0.000	0.600	0.000	994
LLM only	0.113	0.105	0.333	0.444	0.000	0.000	107
Full context	0.962	0.944	0.027	0.000	1.000	0.000	2722

Statistical testing indicates that Workflow GraphRAG demonstrates a statistically significant improvement over custom entity-graph GraphRAG in key-fact recall (+0.257 [0.133, 0.395], Holm-adjusted $p = 0.034$) and ROUGE-L (+0.077 [0.031, 0.127], Holm-adjusted $p = 0.038$), while step coverage differences are not statistically significant after adjustment ($p = 0.438$).

When compared against Naive RAG, Workflow GraphRAG demonstrates an absolute improvement of +0.243 [0.114, 0.381] in key-fact recall. However, under Holm adjustment across the metric family, the difference yields $p = 0.053$. Because $p > 0.05$, we report this comparison as not reaching formal statistical significance at $\alpha = 0.05$. The unadjusted bootstrap confidence interval excludes zero, indicating a consistent empirical direction, but the sample size ($n = 40$) limits conclusive statistical separation under family-wise error control.

Topological inversion rates (TIR) and actor mismatch rates (AMR) are detailed in Table 6. For steps actually detected, Workflow GraphRAG, Naive RAG, Selection-Matched Text RAG, and the supplementary RAG baselines achieved an observed TIR of 0.000, whereas the custom entity-graph baseline and full-context prompting exhibited non-zero inversion rates of 0.083 [0.000, 0.250] and 0.027 [0.000, 0.082], respectively (with the LLM-only reference at 0.333 [0.000, 0.667]). In actor attribution, all retrieval-augmented configurations achieved an observed AMR of 0.000 on detected steps where an actor was explicitly named, while LLM-only exhibited 0.444. These values must be interpreted jointly with step coverage: low inversion rates reflect that when steps are detected, they appear in order, but the primary performance separation

among systems centers on procedural completeness and factual recall rather than reordering of generated steps.

Categorical breakdowns (Table 7) indicate that Workflow GraphRAG achieves strong factual recall across procedural inquiries (1.000 vs. 0.727 for baselines) and cross-SOP inquiries (0.889 vs. 0.611). In cross-SOP questions requiring identification of signatories across distinct procedures, Workflow GraphRAG's step output fields enable unambiguous attribution, mitigating error E3.

Table 7: Key-fact recall by question category for the primary comparison systems on 40 verified questions. Procedural category also includes step coverage.

System	Procedural (KFR/Cov.)	Actor	Documents	Time/location	Cross-SOP
Naive RAG	0.727 / 0.727	0.857	0.619	0.679	0.611
GraphRAG (entity)	0.727 / 0.687	0.786	0.619	0.679	0.611
Workflow GraphRAG	1.000 / 1.000	0.952	0.905	0.964	0.889

6.2. Selection versus structuring ablation (RQ2)

The comparison between Workflow GraphRAG (0.955 KFR) and Selection-Matched Text RAG (0.962 KFR; Table 6) addresses RQ2. The two systems share the identical SOP selection mechanism (Eq. (1)) and top-3 supporting chunk retrieval, differing only in whether the primary procedure is supplied as a structured graph block or as raw PDF text.

A paired Wilcoxon signed-rank test on answerable questions ($n = 35$) yields a paired mean difference of -0.007 $[-0.036, 0.021]$ with $p = 0.59$. Factual scores differ on only 3 of 35 questions. This empirical test does not detect a statistically significant difference between graph-structured context and selection-matched text context. We emphasize that $p = 0.59$ does not prove equivalence between the two representations; rather, the data provide no evidence that explicit graph serialization confers an incremental factual accuracy advantage over supplying the selected procedure's raw text on this benchmark.

The apparent difference in step coverage (1.000 for Workflow GraphRAG vs. 0.975 for Selection-Matched Text RAG) traces to a single procedural item (UKT-P1). As identified in Section 5.6, the ablation system formatted actor and action on distinct markdown lines, preventing the line-based deterministic scorer from registering step co-occurrence despite all 11 steps being present in the generated text.

Regarding selection performance, exactly 36 items in the benchmark assign a single procedure in their metadata (32 answerable single-SOP questions and 4 unanswerable questions tied to specific procedural topics, namely NA-1, NA-2, NA-3, NA-5). The remaining 4 items comprise 3 cross-SOP questions (X-1, X-2, X-3) and 1 non-SOP unanswerable question (NA-4). For all 32 single-SOP answerable questions, the selection rule identified the correct single procedure. For the 4 unanswerable single-SOP items, the selector identified the topical procedure for 3 items (NA-1, NA-2, NA-3), allowing the generator to produce evidence-grounded refusals, and returned no match for 1 item (NA-5), triggering

deterministic refusal. Across all 36 single-procedure items, the selector operated without misrouting between procedures.

Supplementary Ragas evaluations align with this finding: `llama3.1:8b` rates Workflow GraphRAG at 0.805 faithfulness and 0.682 relevancy, compared to 0.762/0.549 for Naive RAG and 0.743/0.537 for GraphRAG, confirming directional alignment.

6.3. Refusal and boundary handling (RQ3)

Across the 5 unanswerable benchmark questions, Workflow GraphRAG, Selection-Matched Text RAG, and Full-Context Prompting achieve 1.000 refusal accuracy with 0.000 false refusals on answerable items (Table 6). LLM-only generation exhibits 0.000 refusal accuracy and 0.000 false refusals, as the zero-shot model attempts to answer every prompt without abstaining (the value of 0.333 appearing in earlier draft tables was an editorial column transposition of its Topological Inversion Rate, now corrected).

Naive RAG, custom entity-graph GraphRAG, Hybrid RAG, Agentic RAG, and Multimodal-Layout RAG each report 0.600 refusal accuracy and 0.000 false refusal rate. A forensic audit of the evaluation logs clarifies the computational accounting behind these figures. In Naive RAG, queries with maximum vector similarity below $\tau = 0.30$ trigger a deterministic pre-generation refusal, returning a fixed Indonesian fallback template ("*Maaf, saya tidak menemukan jawaban pada dokumen SOP resmi yang tersedia*"). This occurred on exactly 10 queries (8 answerable and 2 unanswerable: NA-4 and NA-5), explaining the $30/40 = 0.75$ mean LLM calls per query. However, the evaluation regex patterns in the benchmark specification checked for phrases such as "*tidak dapat menemukan*" and "*tidak bisa menemukan*" while omitting the exact substring "*tidak menemukan*". Consequently, the automated scorer classified the deterministic response as a non-refusal (*refused* = False). For the 8 answerable queries, this resulted in KFR = 0.000 and false_refusal = 0.000; for the 2 unanswerable items, it resulted in failed refusals (refusal_correct = 0.0). Only the remaining 3 unanswerable queries (NA-1, NA-2, NA-3), which invoked the LLM and generated recognized abstention keywords ("*tidak ditemukan*"), were credited as successful refusals ($3/5 = 0.600$). Corrective RAG achieves 1.000 refusal accuracy but incurs a 0.371 false refusal rate (13/35) due to its aggressive confidence threshold ($\tau = 0.35$).

Importantly, Workflow GraphRAG's refusal behavior combines pre-generation filtering with model generation. Two unanswerable questions (NA-4 and NA-5) were refused deterministically before inference because no SOP score reached 3 and vector similarity was below 0.3 (accounting for $38/40 = 0.95$ LLM calls per query). For the remaining three unanswerable questions (NA-1, NA-2, NA-3), an SOP topic was matched by keyword rules, passing context to the generator; the generator successfully produced abstention statements (e.g., stating that fee or GPA criteria were not found in the SOP). Under this architecture, if an unanswerable question mentions SOP keywords, it proceeds to generation rather than filtering deterministically. While effective on the

evaluated benchmark items, this represents an operational boundary rather than generalizable evidence sufficiency verification.

6.4. Computational efficiency and resource trade-offs (RQ4)

Table 8 summarizes system efficiency across token consumption, generator calls, and median latency. Workflow GraphRAG consumes an average of 1,525 prompt tokens per query across the 40 benchmark questions. This represents an increase over Naive RAG (916 tokens) and custom entity-graph GraphRAG (949 tokens) due to serializing the complete workflow block. However, compared to Full-Context Prompting (2,722 tokens), Workflow GraphRAG achieves a 44% prompt-token reduction (operating at ≈56% of full-context volume) while attaining numerically close key-fact recall (0.955 vs. 0.962).

Table 8: Operational resource consumption per query across evaluated systems: mean prompt and completion tokens, mean generator invocations, and median execution latency across all 40 questions.

System	Prompt tok.	Completion tok.	LLM calls	Latency (s)
Naive RAG	916	555	0.75	9.9
GraphRAG (entity)	949	559	0.75	9.6
Workflow GraphRAG	1525	710	0.95	10.7
Hybrid RAG	882	566	0.75	9.3
Agentic RAG	1815	1857	3.27	28.7
Corrective RAG	725	661	1.05	12.7
Multimodal-layout RAG	994	561	0.75	9.8
LLM only	107	792	1.00	10.4
Full context	2722	792	1.00	12.1

The mean number of generator invocations per query reflects deterministic refusal logic: Naive RAG and chunk-based baselines refuse queries with cosine similarity below $\tau = 0.3$ without invoking the generator (evaluating 30 of 40 queries, yielding a mean of $30/40 = 0.75$ calls per query). Workflow GraphRAG executes deterministic refusal only when no SOP score reaches 3 and vector similarity is below 0.3 (refusing 2 of 40 queries prior to inference, yielding $38/40 = 0.95$ calls per query). Full-context prompting evaluates all 40 queries directly ($40/40 = 1.00$ calls). In contrast, Agentic RAG invokes the generator iteratively within its ReAct search loop, averaging 3.27 calls per query.

Execution latency remains practical: Workflow GraphRAG exhibits a median latency of 10.7 seconds on local hardware, comparable to Naive RAG (9.9 s) and Full-Context Prompting (12.1 s). In contrast, Agentic RAG requires multiple ReAct iterations, resulting in 1,815 prompt tokens, 1,857 completion tokens, and a median latency of 28.7 seconds without improving factual accuracy (0.712 KFR).

We distinguish prompt-token savings from total operational expenditures: local deployment eliminates per-token API charges, but requires dedicated hardware memory and compute resources, which were not financially measured.

6.5. Embedding sensitivity ablation

To examine retrieval sensitivity, we repeated evaluations of Naive RAG, GraphRAG, and Workflow GraphRAG using the non-fine-tuned base embedding model (`all-indo-e5-small-v4` base) instead of the fine-tuned checkpoint (Table 9).

Table 9: Ablation evaluating fine-tuned (v3) versus non-fine-tuned base embedding representations: key-fact recall and step coverage across primary systems.

System	v3 embedding	Base embedding
Naive RAG	0.712 / 0.727	0.910 / 0.898
GraphRAG (entity)	0.698 / 0.687	0.900 / 0.865
Workflow GraphRAG	0.955 / 1.000	0.955 / 1.000

Under the base embedding model, Naive RAG performance increases substantially from 0.712 to 0.910 KFR (and step coverage from 0.727 to 0.898). Similarly, custom entity-graph GraphRAG increases from 0.698 to 0.900 KFR. Workflow GraphRAG remains consistent at 0.955 KFR.

This reveals substantial sensitivity to embedding configuration. Replacing the fine-tuned checkpoint with its base counterpart substantially improved the retrieval and recall performance of conventional chunk-based baselines, while Workflow GraphRAG remained consistent at 0.955 KFR. Although overfitting to the phrasing of the 50 fine-tuning question-answer pairs is a plausible hypothesis, the present experiments do not establish the definitive mechanism behind this degradation. Alternative factors include objective mismatch between contrastive loss and administrative chunk retrieval, training instability on small sample volumes, or distortions in embedding-space geometry. Regardless of the underlying cause, this comparison demonstrates that well-configured standard retrieval baselines perform competitively on this corpus (0.910 KFR), substantially narrowing the performance margin between conventional chunk retrieval and workflow-guided methods.

6.6. Qualitative error analysis

Analysis of generated outputs reveals specific behavioral differences across architectures:

1. **Cross-SOP Disambiguation (Question X-2):** For inquiry "Who signs the academic leave permit and late tuition permit letters, respectively?", Naive RAG retrieved chunks from both procedures but failed to map officials to specific documents, listing multiple candidate signatories under each heading (KFR = 0.5). Workflow GraphRAG leveraged explicit step output attributes to assign signatories unambiguously ("Dean for Academic Leave Permit; Vice Dean for Resources for Late Tuition Permit"), achieving KFR = 1.0.

2. **Omission of Conditional Qualifications (Question IRS-A1):** In course registration inquiry "Who approves the IRS form and how?", the official procedure specifies that the academic advisor approves online via SIAP once

credit loads are verified. Workflow GraphRAG correctly identified the advisor and SIAP portal, but omitted the prerequisite credit-load condition, scoring KFR = 0.667.

3. **Formatting Artifacts in Step Scoring (Question UKT-P1):** In late tuition payment, the selection-matched text ablation outputted step actors and actions on separate markdown lines. While complete and correct upon human inspection, the line-segmented automatic scorer registered partial step detection, illustrating measurement sensitivity to generator output formatting.

7. Discussion and threats to validity

7.1. Empirical findings and architectural interpretation

The empirical results establish two distinct conclusions:

1. **Demonstrated Finding:** The complete Workflow GraphRAG system, combining rule-based procedure selection and structured workflow blocks, achieves high factual completeness (0.955 KFR) and step coverage (1.000) on the institutional benchmark, attaining factual recall numerically close to full-context prompting while utilizing 44% fewer prompt tokens.

2. **Non-Demonstrated Finding:** Explicit graph-derived workflow serialization was *not* demonstrated to provide a statistically detectable factual accuracy advantage over providing the selected procedure's raw text ($p = 0.59$).

This indicates that in concise, linear administrative workflows, procedure-level selection is an essential differentiator compared to unconstrained passage retrieval. When standard top-k retrieval injects three competing procedures into the prompt, the small model suffers procedural cross-contamination. Resolving procedure selection—whether via rule-based routing, metadata filtering, or workflow indexing—mitigates this failure mode. However, the current experimental design does not fully isolate selection's causal contribution from accompanying query-expansion and context-budgeting variations. The empirical value of explicit graph structuring in this setting lies in standardized output formatting, concise prompt presentation, and explicit deliverable attribution, rather than an incremental gain in factual recall.

7.2. Threats to internal and comparative validity

We transparently identify key threats to validity:

- **Custom Baseline Implementation:** The entity-graph GraphRAG baseline was constructed by the authors using 17 hand-curated relational triples. It does not represent published, automated frameworks such as Microsoft GraphRAG [3] or LightRAG [12]. While the observed comparison ($p = 0.034$) is valid against this custom baseline, it should not be construed as a generalized claim against the GraphRAG paradigm.

- **Author Benchmark Curation and Verification:** Benchmark questions, reference answers, and gold step annotations were developed and verified by the first author against official PDFs. Although verified against source documents, verification was not conducted by an independent administrative panel, introducing potential alignment between benchmark design and evaluated capabilities.
- **Sample Size and Multiplicity Constraints:** The benchmark comprises 40 verified questions. The comparison between Workflow GraphRAG and Naive RAG yielded a borderline adjusted result ($p = 0.053$). While bootstrap confidence intervals suggest positive separation, statistical power is constrained at $n = 40$.
- **Scope of Scorer Audit:** The manual scorer audit validated 60 sampled rows across three systems, confirming 0.983 agreement and zero actor errors in that sample. This audit establishes scorer reliability, but cannot be extrapolated to assert zero actor errors across all uninspected outputs.

7.3. Corpus and generalizability limitations

- **Linear, Short Procedures:** The corpus comprises 7 one-page linear procedures totaling 1,097 words. These workflows contain no conditional branching, parallel executions, or decision gateways. While representative of institutional student services, findings cannot be generalized directly to complex, branching enterprise workflows where formal BPMN gateways may prove essential.
- **Rule-Based Selection Transferability:** The selection mechanism utilizes keyword and alias heuristics tailored to this 7-SOP collection. In large-scale corpora containing hundreds of procedures, manual alias curation is impractical; automated classification or hierarchical vector routing would be required.
- **Supporting Chunk Contamination:** Appending top-3 vector chunks after the workflow block reintroduces passages from unselected SOPs. Future ablations should isolate the impact of removing or filtering supporting chunks to strictly within-SOP passages.
- **Single Generator Architecture:** Evaluations were conducted on a single local small model (`gemma4:e2b`) at temperature 0. Evaluating across additional model families (e.g., Llama-3.2, Qwen-2.5) is necessary to determine model invariance.

8. Conclusion and future work

This study investigated the empirical relationship between Standard Operating Procedure selection and explicit workflow graph structuring in local retrieval-augmented question answering. Evaluating across 10 system configurations on 40 verified institutional questions using a locally hosted small generator (`gemma4:e2b`), we addressed our core research questions:

- **RQ1 (Answer Quality):** Workflow GraphRAG achieved 0.955 key-fact recall and 1.000 step coverage, improving upon naive chunk retrieval (0.712 KFR, Holm $p = 0.053$) and custom entity-graph retrieval (0.698 KFR, Holm $p = 0.034$). In step order preservation, Workflow GraphRAG achieved an observed TIR of 0.000, whereas custom entity-graph RAG and full-context prompting exhibited non-zero inversion rates of 0.083 and 0.027.
- **RQ2 (Selection vs. Structuring):** A controlled selection-matched text ablation achieved 0.962 KFR. A paired Wilcoxon test detected no statistically significant difference between graph-structured context and selected raw text ($p = 0.59$), confirming that explicit graph serialization yielded no incremental factual recall benefit over providing the selected procedure's raw text on this benchmark.
- **RQ3 (Refusal):** The system achieved 1.000 refusal accuracy on 5 unanswerable questions through a combination of selection-threshold filtering and generated refusals.
- **RQ4 (Efficiency):** Workflow GraphRAG consumed 1,525 prompt tokens per query, attaining factual recall numerically close to full-context prompting while reducing prompt tokens by 44%.

For institutional practitioners deploying small language models over concise procedural documents, our findings emphasize the practical value of procedure-level selection for reducing cross-document distractor interference. Explicit workflow graphs provide structured output consistency, guarantee chronological ordering, and reduce prompt tokens relative to full context, but do not automatically increase factual recall over well-selected raw text.

Future work will focus on: (1) replacing heuristic rule selection with learned dense procedural classifiers; (2) scaling workflow representations to branching procedures by parsing official BPMN XML diagrams; (3) evaluating against published open-source GraphRAG implementations (LightRAG, HippoRAG); and (4) conducting an a priori powered evaluation with independent administrative validation.

Acknowledgements

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors. The authors thank the Dean of the Faculty of Science and Mathematics, Universitas Diponegoro, Prof. Dr. Kusworo Adi, the Head of the Department of Informatics, Dr. Aris Sugiharto, the Undergraduate Thesis Coordinator, Nurdin Bahtiar, and the thesis examiners, Dr. Sutikno and Yunila Dwi Putri Ariyanti, for their guidance and feedback, and the Faculty of Science and Mathematics for permitting this research to be carried out using its SOPs.

Data and code availability

Code, benchmark, and evaluation scripts are available at `https://github.com/reon01425-glitch/Chatbot-RAG-Comparison`, branch `exp/full-run-v1`. Experimental logs are deposited in `results/full_v3_20261008/` (main run) and `results/full_base_20261008/` (embedding ablation). The repository includes the 7 source SOP documents redistributed under institutional permission.

CRediT authorship contribution statement

Cipta Fikri Wiratama: Conceptualization, Methodology, Software, Data curation, Validation, Investigation, Writing – original draft. **Indra Waspada**: Conceptualization, Supervision, Writing – review & editing. **Dhena Kamalia Fu'adi**: Data curation, Validation, Writing – review & editing. **Ramona Ramli**: Conceptualization, Methodology, Supervision, Writing – review & editing. **Henri Tantyoko**: Methodology, Validation, Writing – review & editing. **Sandy Kurniawan**: Supervision, Methodology, Writing – review & editing. **Satriawan Rasyid Purnama**: Validation, Formal analysis, Writing – review & editing.

Declaration of competing interest

The authors declare no competing interests.

Declaration of generative AI use

During the preparation of this work, the authors used generative AI tools to assist with productivity in literature search, drafting, and academic language editing. All methodological formulations, system implementations, and experimental analyses were designed by the authors, who reviewed and edited all generated text and assume full responsibility for the published content.

Appendix A. Full results with confidence intervals

Table A.10 presents the complete evaluation results across all 10 systems, reporting mean values alongside 95% bootstrap confidence intervals (10,000 resamples).

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 2020, pp. 9459–9474.

Table A.10: Complete evaluation results across all 10 systems on 40 verified benchmark questions, reporting mean [95% bootstrap confidence intervals].

System	KFR ↑	Coverage ↑	TIR ↓	AMR ↓	Refusal acc. ↑	False ref. ↓	Prompt tok.
Naive RAG	0.712 [0.571, 0.840]	0.727 [0.455, 1.000]	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	0.600 [0.200, 1.000]	0.000 [0.000, 0.000]	916
GraphRAG (entity)	0.698 [0.560, 0.829]	0.687 [0.415, 0.920]	0.083 [0.000, 0.250]	0.000 [0.000, 0.000]	0.600 [0.200, 1.000]	0.000 [0.000, 0.000]	949
Workflow GraphRAG	0.955 [0.917, 0.990]	1.000 [1.000, 1.000]	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	1.000 [1.000, 1.000]	0.000 [0.000, 0.000]	1525
Selection-Matched Text RAG (ablation)	0.962 [0.914, 1.000]	0.975 [0.926, 1.000]	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	1.000 [1.000, 1.000]	0.000 [0.000, 0.000]	1479
Hybrid RAG	0.683 [0.536, 0.819]	0.712 [0.439, 0.970]	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	0.600 [0.200, 1.000]	0.000 [0.000, 0.000]	882
Agentic RAG	0.712 [0.571, 0.840]	0.702 [0.430, 0.950]	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	0.600 [0.200, 1.000]	0.000 [0.000, 0.000]	1815
Corrective RAG	0.569 [0.414, 0.719]	0.636 [0.364, 0.909]	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	1.000 [1.000, 1.000]	0.371 [0.229, 0.543]	725
Multimodal-layout RAG	0.702 [0.562, 0.831]	0.702 [0.430, 0.950]	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	0.600 [0.200, 1.000]	0.000 [0.000, 0.000]	994
LLM only	0.113 [0.050, 0.183]	0.105 [0.000, 0.209]	0.333 [0.000, 0.667]	0.444 [0.000, 1.000]	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	107
Full context	0.962 [0.914, 1.000]	0.944 [0.871, 1.000]	0.027 [0.000, 0.082]	0.000 [0.000, 0.000]	1.000 [1.000, 1.000]	0.000 [0.000, 0.000]	2722

[2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, H. Wang, Retrieval-augmented generation for large language models: A survey, arXiv:2312.10997 (2023).

[3] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, J. Larson, From local to global: A graph RAG approach to query-focused summarization, arXiv:2404.16130 (2024).

[4] B. Peng, Y. Zhu, Y. Liu, X. Bo, H. Shi, C. Hong, Y. Zhang, S. Tang, Graph retrieval-augmented generation: A survey, arXiv:2408.08921 (2024).

[5] S. Robertson, H. Zaragoza, The probabilistic relevance framework: BM25 and beyond, Foundations and Trends in Information Retrieval 3 (4) (2009) 333–389.

[6] G. V. Cormack, C. L. A. Clarke, S. Büttcher, Reciprocal rank fusion outperforms Condorcet and individual rank learning methods, in: Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, 2009, pp. 758–759.

[7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations (ICLR), 2023.

[8] S.-Q. Yan, J.-C. Gu, Y. Zhu, Z.-H. Ling, Corrective retrieval augmented generation, arXiv:2401.15884 (2024).

[9] A. Asai, Z. Wu, Y. Wang, A. Sil, H. Hajishirzi, Self-RAG: Learning to retrieve, generate, and critique through self-reflection, in: International Conference on Learning Representations (ICLR), 2024.

[10] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang, Lost in the middle: How language models use long contexts, Transactions of the Association for Computational Linguistics 12 (2024) 157–173.

[11] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, X. Wu, Unifying large language models and knowledge graphs: A roadmap, IEEE Transactions on Knowledge and Data Engineering 36 (7) (2024) 3580–3599.

[12] Z. Guo, L. Xia, Y. Yu, T. Ao, C. Huang, LightRAG: Simple and fast retrieval-augmented generation, arXiv:2410.05779 (2024).

[13] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, Y. Su, HippoRAG: Neurobiologically inspired long-term memory for large language models, in: Advances in Neural Information Processing Systems 37 (NeurIPS 2024), 2024.

[14] L. Lin, Z. Zhu, T. Zhang, Y. Wen, SOPRAG: Multi-view graph experts retrieval for industrial standard operating procedures, arXiv:2602.01858 (2026).

[15] K. Wu, C. Kuai, Z. Li, J. Jiang, S. Shen, S. Wang, C.-W. Hu, Z. Tu, Y. Zhou, Knowledge is not static: Order-aware hypergraph RAG for language models, arXiv:2604.12185 (2026).

[16] M. Dumas, M. La Rosa, J. Mendling, H. A. Reijers, Fundamentals of Business

Process Management, 2nd Edition, Springer, 2018.

[17] Object Management Group, Business Process Model and Notation (BPMN), Version 2.0, OMG Standard formal/2011-01-03 (2011). Also published as ISO/IEC 19510:2013.

[18] F. Friedrich, J. Mendling, F. Puhlmann, Process model generation from natural language text, in: Advanced Information Systems Engineering (CAiSE 2011), Vol. 6741 of Lecture Notes in Computer Science, Springer, 2011, pp. 482–496.

[19] LazarusNLP, all-indo-e5-small-v4, Hugging Face model card, `https://huggingface.co/LazarusNLP/all-indo-e5-small-v4`, accessed May 2026.

[20] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, F. Wei, Text embeddings by weakly-supervised contrastive pre-training, arXiv:2212.03533 (2022).

[21] M. Henderson, R. Al-Rfou, B. Strope, Y.-H. Sung, L. Lukács, R. Guo, S. Kumar, B. Miklos, R. Kurzweil, Efficient natural language response suggestion for smart reply, arXiv:1705.00652 (2017).

[22] C.-Y. Lin, ROUGE: A package for automatic evaluation of summaries, in: Text Summarization Branches Out, Association for Computational Linguistics, 2004, pp. 74–81.

[23] S. Es, J. James, L. Espinosa-Anke, S. Schockaert, RAGAs: Automated evaluation of retrieval augmented generation, in: Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, 2024, pp. 150–158.

[24] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, I. Stoica, Judging LLM-as-a-judge with MT-Bench and Chatbot Arena, in: Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track, 2023.

[25] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall/CRC, 1993.

[26] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.

[27] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.